

Đào Thịnh Phú:

báo cáo: [Bao cao tong hop he goi y Amazon Vietnam LLM CHGNN](#)

code : [MMDv3_improved.ipynb](#)

Họ và tên: Đào Văn Công

Báo cáo về xử lý phân tán sử dụng Spark trên Google Cloud.

Áp dụng Training phân tán để huấn luyện các models với tập dữ liệu lớn.

Báo cáo về quy trình ETL với Spark cho tập dữ liệu lớn trên Google Cloud

Mục tiêu: Xử lý ổn định, tránh tràn RAM, tăng tốc độ/xử lý và tối ưu chi phí hạ tầng Cloud.

Áp dụng vào các file code: *etl_item_nodes_v2.py*, *etl_interactions_v2.py*,
evaluation_dataset_v2.py, *data_validator_v2.py*

1. Tóm tắt điều hành (Executive Summary)

Pipeline ETL hiện tại được thiết kế theo hướng xử lý phân tán: đọc JSONL bằng schema đã thiết lập để chuẩn hóa dữ liệu từ nhiều nguồn về 1 dạng schema chung, ghi kết quả trung gian sang Parquet, sau đó tạo bộ evaluation ở dạng ID-only.

Cách xử lý của nhóm hợp với dữ liệu khoảng 55GB vì hạn chế suy luận schema, tránh kéo toàn bộ dữ liệu về khiến tràn RAM, tận dụng xử lý phân tán, và giảm dung lượng đọc lại ở các bước sau.

Điểm mạnh nổi bật là các bước xử lý đều ưu tiên select cột cần thiết, chuẩn hóa bằng hàm Spark thay vì Python loop, và tránh join các cột chứa dữ liệu text lớn trong bước tạo negative candidates.

2. Quy trình ETL hiện tại

2.1. Chuẩn hóa dữ liệu sản phẩm (*etl_item_nodes_v2.py*)

Bước item nodes đọc metadata từ file JSONL của VN và Amazon bằng schema định nghĩa sẵn.

- Dữ liệu đầu vào:

dữ liệu nguồn việt nam: VN (product_id, asin, name, specs, desc, bc).

dữ liệu nguồn amazon: Amazon (parent_asin, asin, title, features, desc, details).

- Kỹ thuật tối ưu:

Schema chung: Spark không phải scan dữ liệu để đoán schema, giảm I/O tới Cloud Storage.

Ghi kết quả theo dạng parquet, phân ra từng *coalesce(16)*, khử trùng theo *product_id* + *asin*.

2.2. Chuẩn hóa dữ liệu tương tác/review

- Quy trình: Đọc review VN/Amazon, map các trường tương ứng, *unionByName*, loại dòng trống.
- Kỹ thuật tối ưu:
 - + Khử trùng theo *user_id* + *product_id* làm giảm kích thước downstream.
Bước này làm bùng nổ dữ liệu
 - + Ghi Parquet trung gian giúp các bước sau không phải đọc lại JSONL 55GB gốc.

2.3. Tạo evaluation dataset

- Chỉ chọn các ID: chỉ select ID, ASIN, Category để phục vụ Positive pairs khớp theo ASIN.
- Negative Mining: Tách thành hard negatives (cùng category) và easy negatives (random).
- Kỹ thuật tối ưu:
 - + Giới hạn Hard pool (~500 item/cat) và Easy pool (~2000 item) trước khi Join để tránh bùng nổ dữ liệu hoặc mắc kẹt với các loại hàng hóa chiếm số đông xét theo category.
 - + Không join metadata text lớn: hướng làm này giúp tránh RAM tăng đột biến bởi vì bước Join này chiếm 90% dung lượng.

3. Xử lý dữ liệu 55GB và tránh tràn RAM (OOM)

- Vài nguyên nhân như đọc 1 thể luôn cả 55GB, và lúc thực thi Spark có các hàm như *dropDuplicates* khiến cho lượng Memory dự kiến - pending cần chờ xử lý nhảy lên đến 1 TiB với dữ liệu sản phẩm và 3 TiB với dữ liệu review sản phẩm.

Rủi ro	Cách xử lý trong V2	Tác động
Spark phải đoán schema JSONL	Định nghĩa Schema (StructType) trước khi đọc	Giảm scan đầu vào, giảm thời gian và I/O
Cross join làm nổ dữ liệu	Giới hạn hard/easy negative pool trước khi broadcast	Kiểm soát số record sinh ra, tránh OOM
Cache tràn memory executor	Dùng MEMORY_AND_DISK, unpersist sau khi dùng	Spill dữ liệu đã nén sang disk thay vì fail job
Đọc quá nhiều cột text	Select cột cần thiết, evaluation lưu ID-only	Giảm RAM, shuffle và chi phí đọc

Rủi ro	Cách xử lý trong V2	Tác động
Quá nhiều file nhỏ	Coalesce/repartition trước khi ghi (mục tiêu 256-512MB/file)	Giảm overhead request và planning

4. Tối ưu tốc độ đọc và xử lý

- Định dạng file Parquet/Snappy: dạng file parquet cho phép đọc theo cột, giảm byte lúc read khi chỉ cần lấy các cột cần thiết.
- Tránh Python UDF: Vì chuyển đổi chờ đợi giữa Python và JVM khiến đợi lâu và có khi lỗi vì Python đọc dữ liệu quá to thì không xử lý được. Nhóm sử dụng các hàm Spark SQL (*regexp_replace*, *concat_ws*) để Spark Catalyst Optimizer có thể can thiệp tối ưu luôn khi chạy JVM.
- Kiểm soát Skew - phân bố dữ liệu: chọn cách lấy số lượng (≤ 1 mức) của các category có số sản phẩm quá lớn để tránh tình trạng 1 task chạy mãi không xong. Ví dụ như 1 loại mặt hàng có quá nhiều sản phẩm trong khi các loại mặt hàng khác quá ít sản phẩm. Nếu không xử lý được sẽ treo máy rất lâu khi chạy Evaluation
- Spark có cơ chế 1 task bị quá tải trên 1 VM, sẽ được 1 VM khác đang rảnh xử lý.

5. Giảm chi phí trên Google Cloud Storage (GCS)

- Lưu trữ theo cột: Chỉ đọc cột cần thiết để giảm trực tiếp chi phí Retrieval/Read.
- Hạn chế file nhỏ: Nhiều file nhỏ làm tăng phí Request (List/Get). Mục tiêu file đầu ra 256MB-512MB.
- Vị trí địa lý: Đặt Bucket và Compute (Dataproc) cùng Region để tránh phí Network Transfer và độ trễ.
- Thiết lập Cluster với 3 Node chính, 2 Node Secondary để giảm chi phí

Báo cáo về quy trình Training Phân tán cho dữ liệu lớn trên Google Cloud

- **Mục tiêu:** Huấn luyện mô hình quy mô lớn (55GB dữ liệu) ổn định, tối ưu hóa hiệu suất Multi-GPU, tránh lỗi tràn RAM (OOM) và tự động hóa quy trình triển khai.
- **Phạm vi:** *main.py*, *src/data_utils.py*, *src/gcs_manager.py*, *src/baselines/*, *submit_job.sh*

1. Tóm tắt điều hành (Executive Summary)

Hệ thống huấn luyện được thiết kế theo kiến trúc **Distributed Data Parallel (DDP)** chạy trên nền tảng **GCP Vertex AI Custom Job**. Điểm khác biệt cốt lõi là việc chuyển đổi từ quy trình huấn luyện truyền thống sang quy trình **"Precomputed-First"**:

- Tiền tính toán Embedding: Loại bỏ gánh nặng mã hóa BERT/Transformer trong vòng lặp huấn luyện, giúp GPU tập trung 100% vào việc học các tương tác.
- Quản lý dữ liệu thông minh: Sử dụng Memory-Mapping và PyArrow để xử lý tập dữ liệu khổng lồ mà không cần cấu hình RAM khổng lồ.
- Vận hành theo Workflow "Vàng": Sử dụng Coordinator VM để đóng gói và điều phối, đảm bảo tính nhất quán giữa môi trường phát triển và môi trường Cloud.

2. Kiến trúc Hệ thống Training Phân tán

2.1. Cơ chế Multi-GPU với PyTorch DDP

Hệ thống sử dụng **DistributedDataParallel (DDP)** kết hợp với *torchrun* để thực thi trên cấu hình nhiều GPU (mặc định 4x GPU T4).

- Giao tiếp: Sử dụng backend *nccl* (NVIDIA Collective Communications Library) cho tốc độ đồng bộ gradient tối ưu.
- Điều phối: *torchrun* tự động quản lý các process con, gán *RANK* và *LOCAL_RANK* cho từng GPU, giúp việc chia nhỏ dữ liệu (*DistributedSampler*) diễn ra chính xác.

2.2. Chiến lược Precomputed Embeddings (Tăng tốc 5-10x)

Đây là giải pháp then chốt để vượt qua giới hạn về CPU và GPU:

- Giai đoạn 1 (Precompute): Mã hóa toàn bộ text sang vector một lần duy nhất, lưu thành file *.npy* (khoảng 12GB).
- Giai đoạn 2 (Fast Training): Model chỉ cần thực hiện phép nhân ma trận nhẹ nhàng giữa các vector đã có sẵn.
- Lợi ích: Loại bỏ tình trạng GPU phải "chờ" CPU mã hóa văn bản, giảm thời gian huấn luyện mỗi epoch từ hàng tiếng xuống còn vài phút.

2.3. Module *data_utils.py* giúp quản lý dữ liệu

Module của file này load dữ liệu và tối ưu bộ nhớ, tránh dữ liệu chiếm hết RAM và Disk

- *PrecomputedEmbeddingLookup*: Cơ chế tra cứu vector dùng bảng tra cứu (mapping) trực tiếp từ ID sản phẩm sang index trong mảng Numpy, giảm thiểu độ trễ I/O lúc training.
- Cơ chế Memory-Mapping (*mmap_mode='r'*):
 - + Dữ liệu *.npy* (12GB) là các metadata vector được load vào với chế độ *mmap*, cho phép lấy dữ liệu trực tiếp từ SSD mà không cần tải hết vào RAM.
 - + Tầm quan trọng: Giúp huấn luyện trên các máy ảo RAM nhỏ nhưng vẫn không làm treo hệ thống. Khi train nhiều GPU trên 1 VM, cũng cần vì nó giảm

lượng lấy từ GPU đến RAM vì nếu không có Memory-Mapping thì toàn bộ file .npy sẽ load lên RAM.

- Nạp dữ liệu bằng PyArrow: *pyarrow* để đọc file Parquet.
 - + Chỉ nạp đúng 2 cột cần thiết (*asin*, *product_id*), giảm 90% dung lượng bộ nhớ chiếm dụng so với nạp toàn bộ metadata.
 - + Hỗ trợ *DATA_FRACTION* (ví dụ: 12.5%) giúp nhóm chạy 1 lượng nhỏ dữ liệu mà không cần đợi nạp toàn bộ 55GB dữ liệu thô -> dễ sửa bug
 - + Ban đầu nhóm dùng dataset library bên hugging face, nhưng do nó xét quá chi tiết các cột nên bị lỗi dữ liệu, mặc dù cột đó ko cần lấy nhưng huggingface này quét tất, khi lỗi -> ko chạy

3. Giải pháp Xử lý Dữ liệu Lớn & Tránh tràn RAM (OOM)

Thách thức	Giải pháp kỹ thuật	Hiệu quả
File Vector 12GB gây nổ RAM	Dùng <code>np.load(mmap_mode='r')</code>	Huấn luyện được trên máy ảo RAM nhỏ (16GB)
Tập tương tác 55GB quá lớn	Dùng PyArrow nạp theo tỷ lệ (<i>DATA_FRACTION</i>)	Tăng tốc độ nạp dữ liệu, dễ dàng debug/scale
Treo Process Group khi khởi tạo	Thiết lập <code>dist.barrier()</code> và timeout tường minh	Tránh xung đột và lỗi zombie process trên Cloud
Nạp dữ liệu chậm từ Cloud	Đồng bộ Local SSD bằng <code>gsutil -m</code>	Tối ưu I/O, giảm độ trễ khi training

4. Tối ưu Hạ tầng & Vận hành trên GCP

4.1. Quy trình "Golden Workflow"

Sử dụng mô hình Coordinator VM (e2-standard-4) làm trạm điều khiển trung tâm:

1. Local: Đẩy code lên GitHub.
2. Coordinator: Pull code, build Docker image và push lên Artifact Registry.
3. Vertex AI: Kích hoạt Job với tài nguyên GPU mạnh mẽ, lấy Image từ Registry.
4. Ưu điểm: Tốc độ mạng nội bộ GCP cực nhanh (Docker push chỉ mất vài giây), môi trường containerized sạch 100%.

4.2. Module *gcs_manager.py* giúp load data

Quản lý luân chuyển dữ liệu giữa Local VM và Cloud Storage (GCS) một cách tối ưu:

- Đồng bộ hóa dữ liệu song song: Sử dụng *gsutil -m cp* để tải dữ liệu đa luồng, tận dụng tối đa băng thông nội bộ của GCP.
- Cơ chế Caching:
 - + Hệ thống kiểm tra file vector 12GB đã có trên GCS chưa. Nếu có, nó sẽ tải về thay vì tính toán lại.
 - + Kết quả: Tiết kiệm ít nhất 1 tiếng đồng hồ chờ đợi cho mỗi lượt chạy Job mới.
- Lưu trữ kết quả tự động: Sau khi huấn luyện, model tốt nhất (*best_model.pt*) và bộ nhớ đệm vector sẽ được đẩy ngược lên GCS để phục vụ các lần chạy sau hoặc bước Evaluation.

Hệ thống là 1 pipeline:

- **gcs_manager**: Kéo dữ liệu từ GCS về Local SSD với tốc độ cao.
- **data_utils**: "Ảnh xạ" dữ liệu từ SSD vào Python qua Memory-Mapping (RAM sạch).
- **DDP Training**: Lấy dữ liệu nhanh để đẩy vào GPU, đảm bảo GPU luôn hoạt động.

Tham khảo: Training phân tán: <https://github.com/hkproj/pytorch-transformer-distributed>

Báo cáo phần tìm kiếm ngữ nghĩa
Nguyễn Văn Quý

1. Tóm tắt nội dung thực hiện

Mục tiêu của phần tìm kiếm ngữ nghĩa là xây dựng một pipeline cho phép người dùng nhập truy vấn bằng tiếng Việt và hệ thống trả về các sản phẩm điện tử có nội dung liên quan nhất. Pipeline được thiết kế theo hướng tách biệt giữa nguồn dữ liệu và thuật toán, do đó có thể thử nghiệm trên bộ dữ liệu Amazon quy mô lớn và có thể thay thế bằng dữ liệu sản phẩm tiếng Việt mới hơn khi triển khai thực tế trên website.

Các công việc chính đã thực hiện gồm: hợp nhất và lọc dữ liệu sản phẩm, kiểm tra chất lượng dữ liệu, tạo mô tả văn bản đại diện cho sản phẩm, xây dựng các phương pháp truy hồi gồm BM25, dense retrieval và hybrid search, thử nghiệm truy vấn tiếng Việt, đồng thời lưu các artifact như tập demo, embedding và chỉ mục FAISS để phục vụ web demo.

Hạng mục	Nội dung đã thực hiện	Ý nghĩa
Dữ liệu	Sử dụng dữ liệu sản phẩm Amazon ngành hàng điện tử, gồm metadata, category, mô tả và thông tin kỹ thuật.	Tạo corpus thương mại điện tử quy mô lớn để thử nghiệm pipeline.
Tiền xử lý	Lọc dữ liệu, bỏ sản phẩm thiếu tên, bỏ trùng ASIN, kiểm tra category và null.	Giảm nhiễu trước khi xây hệ thống tìm kiếm.

Biểu diễn sản phẩm	Tạo trường retrieval_text từ title, category, details, description, features và các thông tin liên quan.	Tạo văn bản đầu vào thống nhất cho BM25 và embedding model.
Phương pháp tìm kiếm	Cài đặt BM25, dense retrieval với mô hình đa ngôn ngữ, và hybrid search.	So sánh tìm kiếm từ khóa, tìm kiếm ngữ nghĩa và phương pháp kết hợp.
Chỉ mục vector	Sử dụng FAISS để lưu và tìm kiếm vector sản phẩm.	Tăng tốc truy hồi top-k sản phẩm theo độ gần ngữ nghĩa.
Đánh giá thử nghiệm	Thử một số truy vấn tiếng Việt như laptop học lập trình, tai nghe chống ồn, màn hình chơi game.	Quan sát chất lượng kết quả và xác định hạn chế cần cải thiện.

2. Bối cảnh và phạm vi bài toán

Trong website thương mại điện tử, người dùng thường không chỉ tìm kiếm bằng tên sản phẩm chính xác mà còn nhập các câu truy vấn tự nhiên như “laptop mỏng nhẹ cho sinh viên”, “tai nghe chống ồn”, “điện thoại pin trâu” hoặc “màn hình chơi game 27 inch”. Tìm kiếm từ khóa truyền thống có thể bỏ sót các sản phẩm phù hợp nếu nội dung sản phẩm không trùng chính xác với truy vấn.

Vì vậy, module tìm kiếm ngữ nghĩa được xây dựng để truy hồi sản phẩm dựa trên ý nghĩa của truy vấn và mô tả sản phẩm, thay vì chỉ dựa trên việc khớp từ khóa. Phần hệ thống gợi ý sản phẩm là một module riêng, có thể dùng chung dữ liệu sản phẩm nhưng không phụ thuộc trực tiếp vào pipeline tìm kiếm ngữ nghĩa.

Thành phần	Vai trò trong đề tài
Tìm kiếm ngữ nghĩa	Người dùng nhập query, hệ thống trả về top sản phẩm phù hợp nhất theo từ khóa và ngữ nghĩa.
Hệ thống gợi ý	Gợi ý sản phẩm liên quan dựa trên sản phẩm đã xem, rating, review hoặc hành vi người dùng.
Website demo	Tích hợp hai module vào giao diện thương mại điện tử, ưu tiên trải nghiệm người dùng tiếng Việt.

3. Dữ liệu sử dụng và vai trò của Amazon dataset

Bộ dữ liệu Amazon được sử dụng như một corpus thương mại điện tử quy mô lớn để xây dựng và kiểm chứng pipeline xử lý dữ liệu, tạo embedding và truy hồi sản phẩm. Dữ liệu này có ưu điểm là có số lượng sản phẩm lớn, metadata tương đối phong phú, nhiều trường mô tả và thông tin đánh giá. Điều này phù hợp với yêu cầu của môn Mining Big Data vì nhóm có thể thể hiện quá trình xử lý, hợp nhất và khai thác dữ liệu quy mô lớn.

Tuy nhiên, dữ liệu Amazon cũng có hạn chế: nhiều sản phẩm cũ, phần lớn là tiếng Anh, và chưa phản ánh hoàn toàn thị trường thương mại điện tử Việt Nam hiện tại. Vì vậy, nhóm

không xem Amazon dataset là dữ liệu production cuối cùng, mà xem đây là dữ liệu thử nghiệm quy mô lớn để đánh giá pipeline. Khi triển khai thực tế, hệ thống có thể thay thế corpus bằng dữ liệu sản phẩm tiếng Việt mới hơn mà không cần thay đổi kiến trúc chính.

Lý do dùng Amazon dataset	Giải thích
Quy mô lớn	Có đủ số lượng sản phẩm để thể hiện bài toán xử lý dữ liệu lớn.
Có metadata sản phẩm	Có title, category, description, features, details, rating và một số thông tin liên quan.
Phù hợp thử nghiệm thuật toán	Có thể tạo retrieval_text, sinh embedding và xây vector index để so sánh nhiều phương pháp.
Có thể thay thế corpus	Pipeline được thiết kế tổng quát: thay dữ liệu đầu vào, tạo lại retrieval_text và embedding là có thể áp dụng cho dữ liệu mới.

3.1. Kết quả kiểm tra dữ liệu sau khi merge

Sau khi merge và lọc lại, dữ liệu tập trung vào các nhóm ngành hàng điện tử như Cell Phones & Accessories, Computers, All Electronics, Camera & Photo, Home Audio & Theater, Car Electronics và GPS & Navigation. Việc kiểm tra cho thấy nhóm Books đã được loại khỏi trường main_category. Một số trường như price bị thiếu nhiều, do đó price chỉ nên dùng để hiển thị nếu có, không dùng làm tiêu chí chính trong semantic search.

Vấn đề dữ liệu	Cách xử lý / cách trình bày
Sản phẩm cũ	Ghi nhận là hạn chế của dataset; trọng tâm là kiểm chứng pipeline và thuật toán.
Dữ liệu tiếng Anh	Dùng mô hình embedding đa ngôn ngữ để hỗ trợ truy vấn tiếng Việt trên corpus tiếng Anh.
Price thiếu nhiều	Không dùng price làm tiêu chí truy hồi chính; chỉ hiển thị nếu có.
Một số category bị gán nhầm	Ghi nhận là nhiễu dữ liệu; có thể cải thiện bằng rule lọc title hoặc phân loại product type.

4. Kiến trúc pipeline tìm kiếm ngữ nghĩa

Pipeline được xây dựng theo luồng từ dữ liệu thô đến kết quả tìm kiếm trên website. Các bước chính gồm: đọc dữ liệu, làm sạch, tạo retrieval_text, xây dựng BM25, tạo embedding, xây FAISS index, thực hiện dense search, kết hợp hybrid search và lưu artifact phục vụ demo.

Amazon Product Data

- > Lọc và làm sạch dữ liệu
- > Tạo retrieval_text cho từng sản phẩm
- > Xây BM25 index
- > Sinh embedding bằng mô hình đa ngôn ngữ
- > Xây FAISS vector index
- > Truy vấn tiếng Việt

-> BM25 / Dense Search / Hybrid Search
-> Trả về top-k sản phẩm phù hợp

5. Tóm tắt công việc theo nhóm cell trong notebook

Nhóm cell	Công việc chính	Kết quả tạo ra
Cell 1-3	Cài đặt và import thư viện: sentence-transformers, FAISS, BM25, pandas, pyarrow.	Môi trường chạy notebook.
Cell 4-6	Khai báo đường dẫn, đọc schema parquet, map các cột quan trọng như title, category, details, price, asin.	Xác định đúng dữ liệu đầu vào và các trường cần dùng.
Cell 7-8	Đọc các cột cần thiết vào DataFrame, bỏ sản phẩm thiếu title, bỏ trùng ASIN.	DataFrame sản phẩm đã làm sạch cơ bản.
Cell 9-12	Tạo hàm làm sạch text, tạo retrieval_text, lấy mẫu 10.000 sản phẩm và lưu demo_products.parquet.	Tập demo 10k và mô tả đại diện cho từng sản phẩm.
Cell 13-14	Tokenize retrieval_text và xây BM25 search.	Baseline tìm kiếm theo từ khóa.
Cell 15-18	Load model paraphrase-multilingual-mpnet-base-v2, sinh embedding, xây FAISS index và viết hàm dense search.	Dense retrieval model 1.
Cell 19-22	Load model LaBSE, sinh embedding, xây FAISS index và viết hàm dense search.	Dense retrieval model 2 để so sánh.
Cell 23	Xây hybrid search bằng cách kết hợp BM25 score và dense score.	Phương pháp kết hợp giữa từ khóa và ngữ nghĩa.
Cell 24-27	Hiển thị kết quả, chạy thử query tiếng Việt, tạo danh sách query đánh giá.	Kết quả thử nghiệm và tập truy vấn đánh giá.
Cell 28	Lưu embedding, FAISS index và meta.json.	Artifact phục vụ web demo, tránh phải tính lại từ đầu.

6. Các phương pháp tìm kiếm đã triển khai

6.1. BM25 - tìm kiếm theo từ khóa

BM25 được sử dụng làm baseline. Phương pháp này chấm điểm sản phẩm dựa trên mức độ khớp từ khóa giữa truy vấn và retrieval_text. BM25 có ưu điểm là bắt tốt các từ khóa cụ thể, đặc biệt là thông số kỹ thuật như RAM, SSD, Bluetooth, iPhone, RTX hoặc kích thước

màn hình. Tuy nhiên, BM25 không hiểu sâu ý định người dùng và có thể trả về kết quả không đúng ngữ cảnh nếu sản phẩm chứa cùng từ khóa.

6.2. Dense retrieval - tìm kiếm theo ngữ nghĩa

Dense retrieval sử dụng mô hình sentence embedding để biến retrieval_text của sản phẩm và truy vấn của người dùng thành vector. Sau đó, hệ thống dùng FAISS để tìm các sản phẩm có vector gần nhất với vector truy vấn. Phương pháp này giúp truy hồi theo ý nghĩa, không phụ thuộc hoàn toàn vào việc trùng từ khóa.

Nhóm thử nghiệm hai mô hình embedding đa ngôn ngữ: paraphrase-multilingual-mpnet-base-v2 và LaBSE. Việc dùng mô hình đa ngôn ngữ là cần thiết vì truy vấn của người dùng hướng tới tiếng Việt, trong khi dữ liệu Amazon chủ yếu là tiếng Anh và ngành hàng điện tử thường có nhiều thuật ngữ trộn Việt - Anh như laptop, smartphone, RAM, SSD, Bluetooth, OLED.

6.3. FAISS - chỉ mục tìm kiếm vector

FAISS được dùng để lưu và tìm kiếm vector sản phẩm. Sau khi embedding của sản phẩm được chuẩn hóa, notebook sử dụng IndexFlatIP để tìm các vector có độ tương đồng cao nhất với truy vấn. Với quy mô demo 10k sản phẩm, cách này đơn giản và phù hợp để kiểm chứng pipeline.

6.4. Hybrid search - kết hợp từ khóa và ngữ nghĩa

Hybrid search kết hợp ưu điểm của BM25 và dense retrieval. BM25 giúp bắt từ khóa và thông số kỹ thuật chính xác, trong khi dense retrieval giúp hiểu ý nghĩa của truy vấn tự nhiên. Trong notebook, hệ thống lấy top ứng viên từ BM25 và top ứng viên từ dense search, gộp hai danh sách, chuẩn hóa điểm về cùng thang 0-1, sau đó tính điểm cuối bằng công thức có trọng số.

$$\text{final_score} = \alpha * \text{dense_score} + (1 - \alpha) * \text{bm25_score}$$

Trong đó alpha thể hiện mức độ ưu tiên dense retrieval. Ví dụ $\alpha = 0.6$ nghĩa là điểm cuối ưu tiên 60% cho tìm kiếm ngữ nghĩa và 40% cho BM25. Cách này giúp hệ thống vừa bắt chữ, vừa bắt nghĩa.

7. Lý do demo trên tập 10.000 sản phẩm

Notebook dùng tập demo 10.000 sản phẩm để giảm chi phí tính toán và cho phép thử nghiệm nhiều phương pháp trong thời gian giới hạn. Dataset gốc có hơn 2 triệu sản phẩm sau khi lọc ngành hàng điện tử; nếu sinh embedding cho toàn bộ dữ liệu bằng nhiều mô hình như MPNet và LaBSE thì thời gian chạy, RAM và GPU trên Colab sẽ rất lớn.

Việc sử dụng 10k sản phẩm không có nghĩa là thuật toán chỉ áp dụng được cho 10k sản phẩm. Đây là tập prototype để kiểm chứng pipeline và so sánh các phương pháp. Khi có hạ tầng phù hợp, pipeline có thể mở rộng bằng cách sinh embedding theo batch, lưu embedding ra disk và xây FAISS hoặc vector database cho toàn bộ corpus.

Lý do chọn 10k	Giải thích
Giảm chi phí tính toán	Sinh embedding bằng nhiều mô hình là bước nặng nhất.

Dễ thử nghiệm nhiều phương pháp	Có thể chạy BM25, dense model 1, dense model 2 và hybrid trong thời gian ngắn.
Phù hợp Colab	Tài nguyên RAM/GPU có giới hạn, không phù hợp để encode toàn bộ corpus nhiều lần.
Pipeline vẫn mở rộng được	Có thể batch embedding và xây index lớn hơn khi có hạ tầng tốt hơn.

8. Kết quả quan sát và đánh giá ban đầu

Hệ thống đã chạy được các truy vấn tiếng Việt và trả về kết quả theo từng phương pháp. Một số truy vấn thử nghiệm gồm “laptop mỏng nhẹ cho sinh viên”, “điện thoại pin trâu”, “tai nghe chống ồn”, “chuột không dây văn phòng”, “màn hình chơi game”, “loa bluetooth nhỏ gọn” và “bàn phím cơ giá rẻ”.

Kết quả ban đầu cho thấy pipeline hoạt động đúng về mặt kỹ thuật: dữ liệu được đọc, retrieval_text được tạo, BM25 index và FAISS index được xây dựng, query được xử lý và top-k sản phẩm được trả về. Tuy nhiên, cũng xuất hiện một số failure case. Ví dụ truy vấn “laptop mỏng nhẹ cho sinh viên” có thể trả về laptop bag, laptop sleeve hoặc laptop stand do dữ liệu có nhiều phụ kiện chứa từ “laptop”. Đây là hạn chế của việc chưa có bước phân loại product type hoặc rule loại phụ kiện.

Quan sát	Nhận xét
BM25	Bắt tốt từ khóa nhưng dễ nhầm khi nhiều sản phẩm phụ kiện chứa cùng từ.
Dense retrieval	Tìm theo ý nghĩa tốt hơn, nhưng vẫn có thể bị kéo về nhóm sản phẩm liên quan nhưng không đúng ý định.
Hybrid search	Kết hợp cả hai nguồn điểm, phù hợp để cân bằng giữa từ khóa và ngữ nghĩa.
Dữ liệu	Cần tiếp tục xử lý nhiều category và tách sản phẩm chính khỏi phụ kiện để tăng chất lượng.

9. Hạn chế hiện tại

- Dữ liệu Amazon có nhiều sản phẩm cũ, chưa phản ánh hoàn toàn thị trường Việt Nam hiện tại.
- Dữ liệu chủ yếu là tiếng Anh, trong khi website hướng tới người dùng tiếng Việt.
- Một số sản phẩm bị gán category chưa chính xác hoặc có title không phù hợp với ngành hàng.
- Cột price bị thiếu nhiều nên chưa phù hợp để xử lý tốt các truy vấn theo giá.
- Chưa có bước phân loại product type để phân biệt sản phẩm chính và phụ kiện.
- Việc đánh giá mới ở mức thử nghiệm ban đầu, cần thêm bộ query có nhãn relevance để tính Precision@K, Recall@K hoặc nDCG.

10. Hướng phát triển

- Bổ sung dữ liệu sản phẩm tiếng Việt mới hơn, ưu tiên sản phẩm điện tử sản xuất trong 3 năm gần đây.
- Tách product type để phân biệt laptop, điện thoại, tai nghe, màn hình với các phụ kiện như túi, bao da, giá đỡ.
- Tối ưu retrieval_text bằng cách dùng thêm description, features, details và review summary nhưng loại bỏ trường nhiễu.
- Bổ sung reranker để sắp xếp lại top kết quả sau bước truy hồi ban đầu.
- Mở rộng từ 10k lên tập dữ liệu lớn hơn bằng batch embedding và vector database như Qdrant, Milvus hoặc Elasticsearch.
- Xây dựng bộ đánh giá có nhãn relevance để so sánh định lượng giữa BM25, dense retrieval và hybrid search.
- Tích hợp module recommendation với search trên cùng website để tạo trải nghiệm thương mại điện tử hoàn chỉnh.

11. Kết luận

Phần tìm kiếm ngữ nghĩa đã xây dựng được một pipeline đầy đủ từ dữ liệu sản phẩm đến kết quả truy hồi: đọc và kiểm tra dữ liệu, làm sạch, tạo retrieval_text, xây BM25 baseline, sinh embedding bằng mô hình đa ngôn ngữ, tạo FAISS index, thử nghiệm dense retrieval và hybrid search. Pipeline này chứng minh được khả năng triển khai tìm kiếm sản phẩm theo ngữ nghĩa và có thể mở rộng hoặc thay thế nguồn dữ liệu khi chuyển sang website thương mại điện tử tiếng Việt.

Dù còn hạn chế về độ mới của dữ liệu, nhiều category, thiếu price và chưa có product-type filtering, hệ thống đã đặt nền tảng kỹ thuật quan trọng cho module tìm kiếm. Các hạn chế này được ghi nhận rõ và là hướng cải thiện hợp lý trong giai đoạn tiếp theo.